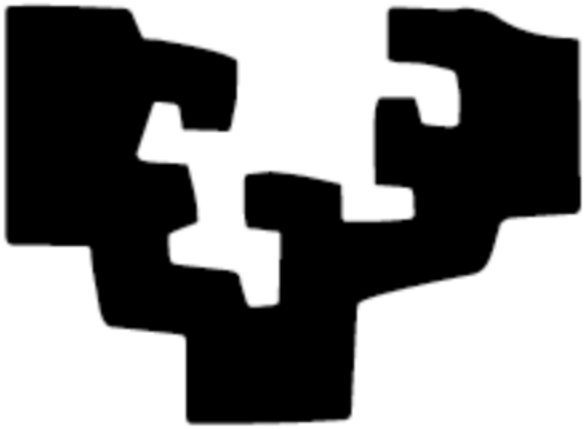


EHU

Proyecto Patrones de Diseño

Luis Pedro Hernández Maldonado

Luis Pedro Figueroa Monroy

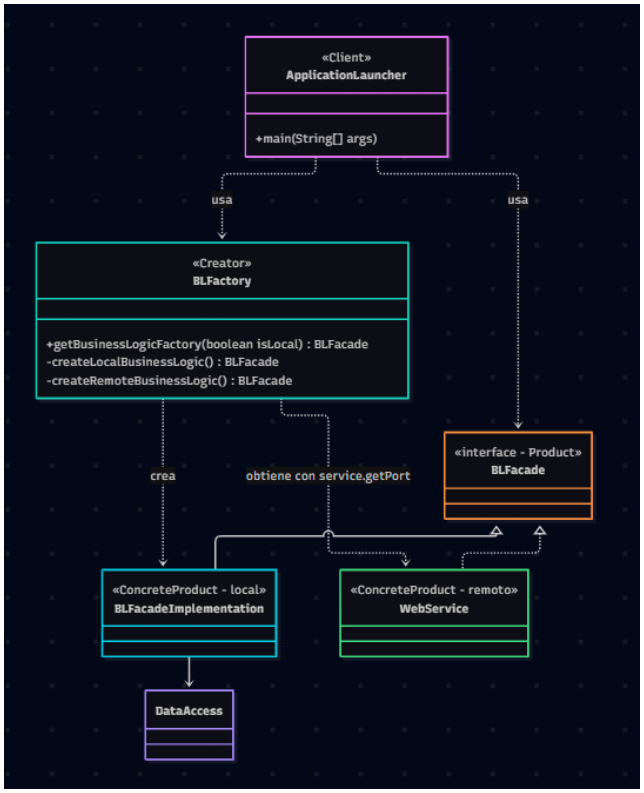
Factory Method

UML y finalidad de cada clase

Antes, ApplicationLauncher decidía qué implementación de la lógica de negocio usar y además la construía. Ahora la creación se centraliza en la factoría BLFactory, y la clase de presentación solo indica cuál quiere.

Papel de cada clase:

- **Creator**: BLFactory. Centraliza la creación del objeto de la lógica de negocio mediante getBusinessLogicFactory(boolean isLocal).
- **Product**: BLFacade. Interfaz que cumplen las dos implementaciones. La presentación solo depende de ella.
- **ConcreteProduct** (local): BLFacadeImplementation. Objeto local que accede a la base de datos a través de DataAccess.
- **ConcreteProduct** (remoto): objeto devuelto por service.getPort(BLFacade.class). Implementa BLFacade y accede a la lógica de negocio mediante servicios web.
- **Client**: ApplicationLauncher. Pide el objeto a la factoría y decide cuál usar según ConfigXML.isBusinessLogicLocal().



(WebService no es una clase, representa lo que se obtiene con service.getPort)

Código antes

```
try {

    BLFacade appFacadeInterface;
    UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");

    if (c.isBusinessLogicLocal()) {
        DataAccess da = new DataAccess();
        appFacadeInterface = new BLFacadeImplementation(da);
    } else { // If remote

        String serviceName = "http://" + c.getBusinessLogicNode() + ":" + c.getBusinessLogicPort() + "/ws/"
                + c.getBusinessLogicName() + "?wsdl";
        URL url = new URL(serviceName);

        // 1st argument refers to wsdl document above
        // 2nd argument is service name, refer to wsdl document above
        QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");

        Service service = Service.create(url, qname);

        appFacadeInterface = service.getPort(BLFacade.class);
    }
```

Código después

```
try {

    UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");

    BLFacade appFacadeInterface = new BLFactory().getBusinessLogicFactory(c.isBusinessLogicLocal());
```

```
/**
 * Factoría que centraliza la obtención del objeto de la lógica de negocio.
 */
public class BLFactory {

    /**
     * Crea la implementación de la lógica de negocio indicada.
     *
     * @param isLocal true para un objeto local, false para un objeto remoto
     * @return el objeto de la lógica de negocio
     */
    public BLFacade getBusinessLogicFactory(boolean isLocal) {
        if (isLocal) {
            return createLocalBusinessLogic();
        }
        return createRemoteBusinessLogic();
    }

    private BLFacade createLocalBusinessLogic() {
        DataAccess da = new DataAccess();
        return new BLFacadeImplementation(da);
    }

    private BLFacade createRemoteBusinessLogic() {
        ConfigXML c = ConfigXML.getInstance();
        String serviceName = "http://" + c.getBusinessLogicNode() + ":" + c.getBusinessLogicPort() + "/ws/"
                + c.getBusinessLogicName() + "?wsdl";

        try {
            URL url = new URL(serviceName);

            QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");

            Service service = Service.create(url, qname);
            return service.getPort(BLFacade.class);
        } catch (MalformedURLException e) {
            throw new IllegalStateException("URL del servicio incorrecta: " + serviceName, e);
        }
    }
}
```

getBusinessLogicFactory(boolean isLocal): según el parámetro, devuelve el objeto local o el remoto. El tipo de retorno es la interfaz BLFacade, de modo que quien la llama no conoce la clase concreta.

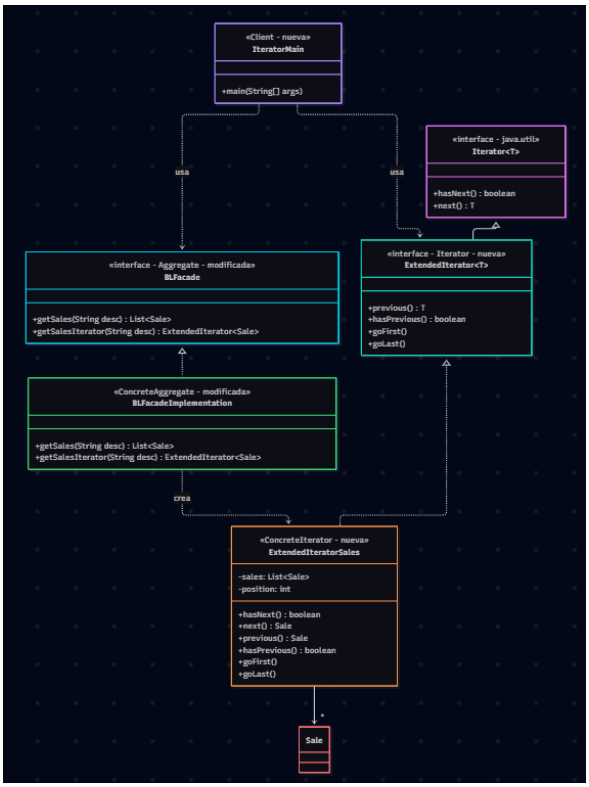
Iterator

UML y finalidad de cada clase

La finalidad de esta sección es añadir a la lógica de negocio el método getSalesIterator(String desc), que hace lo mismo que getSales(String desc) pero devuelve un iterador extendido de ventas en vez de una lista. Este iterador permite recorrer las ventas hacia delante y hacia atrás, y posicionarse en el primer o en el último elemento.

Papel de cada clase:

- **Iterator**: ExtendedIterator<T>. Interfaz que extiende java.util.Iterator<T> con previous, hasPrevious, goFirst y goLast.
- **ConcreteIterator**: ExtendedIteratorSales. Implementa ExtendedIterator<Sale> sobre la lista de ventas. Guarda la lista y la posición actual.
- **Aggregate**: BLFacade. Declara el método que crea el iterador, getSalesIterator(String desc).
- **ConcreteAggregate**: BLFacadeImplementation. Implementa getSalesIterator, creando un ExtendedIteratorSales con las ventas obtenidas.
- **Client**: IteratorMain. Obtiene la lógica de negocio con la factoría de la primera sección y recorre las ventas en los dos sentidos.



Código antes

```java
/**
 * This method retrieves the products that contain desc
 *
 * @param desc the text to search
 * @return collection of sales that contain desc
 */
@WebMethod public List<Sale> getSales(String desc);
```

```java
/**
 * {@inheritDoc}
 */
@WebMethod
public List<Sale> getSales(String desc){
    dbManager.open();
    List<Sale>  rides=dbManager.getSales(desc);
    dbManager.close();
    return rides;
}
```

Antes de las modificaciones solo existía el getSales normal.

Código después

```java
/**
 * Iterador extendido que además de recorrer hacia delante, permite
 * recorrer hacia atrás y posicionarse en el primer o el último elemento.
 */
public interface ExtendedIterator<T> extends Iterator<T> {
    // return the actual element and go to the previous
    public T previous();

    // true if there is a previous element
    public boolean hasPrevious();

    // It is placed in the first element
    public void goFirst();

    // It is placed in the last element
    public void goLast();
}
```

Declara las cuatro operaciones nuevas. Al extender Iterator<T>, el iterador también se puede usar como un iterador normal de Java.

```java
/**
 * Iterador extendido sobre una lista de ventas.
 */
public class ExtendedIteratorSales implements ExtendedIterator<Sale> {
    private List<Sale> sales;
    private int position;

    public ExtendedIteratorSales(List<Sale> sales) {
        this.sales = sales;
        this.position = 0;
    }

    public boolean hasNext() {
        return position < sales.size();
    }

    public Sale next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        return sales.get(position++);
    }

    public boolean hasPrevious() {
        return position >= 0;
    }

    public Sale previous() {
        if (!hasPrevious()) {
            throw new NoSuchElementException();
        }
        return sales.get(position--);
    }

    public void goFirst() {
        position = 0;
    }

    public void goLast() {
        position = sales.size() - 1;
    }
}
```

position indica el elemento actual. next() devuelve el elemento actual y avanza (position++). previous() devuelve el elemento actual y retrocede (position--). goFirst() coloca la posición en 0 (primer elemento) y goLast() en sales.size() - 1 (último elemento).

```java
/**
 * This method retrieves the products that contain desc as an extended iterator
 *
 * @param desc the text to search
 * @return iterator over the sales that contain desc
 */
@WebMethod public ExtendedIterator<Sale> getSalesIterator(String desc);
```

```java
/**
 * {@inheritDoc}
 */
@WebMethod
public ExtendedIterator<Sale> getSalesIterator(String desc) {
    return new ExtendedIteratorSales(getSales(desc));
}
```

Se hace return new ExtendedIteratorSales(getSales(desc)) para no repetir código, el método nuevo reutiliza getSales, que ya abre la base de datos, hace la consulta y la cierra, y solo envuelve la lista resultante en el iterador.

```java
Run | Debug
public static void main(String[] args) {
    // the BL is local
    boolean isLocal = true;
    BLFacade blFacade = new BLFactory().getBusinessLogicFactory(isLocal);

    ExtendedIterator<Sale> it = blFacade.getSalesIterator("a");
    Sale s;
    System.out.println("_____");
    System.out.println("FROM LAST TO FIRST");
    it.goLast(); // Go to last element
    while (it.hasPrevious()) {
        s = it.previous();
        System.out.println(s);
    }
    System.out.println();
    System.out.println("_____");
    System.out.println("FROM FIRST TO LAST");
    it.goFirst(); // Go to first element
    while (it.hasNext()) {
        s = it.next();
        System.out.println(s);
    }
}
```

Ejecución

```
IteratorMain.java
1  package gui;
2
3  import businesslogic.BLFacade;
4  import businesslogic.BLFactory;
5  import businesslogic.ExtendedIterator;
6  import domain.Sale;
7
8  public class IteratorMain {

Problems  Javadoc  Declaration  Console  Coverage  Terminal
<terminated> IteratorMain [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe (Nov 7, 20
Read from config.xml:    businessLogicLocal=true databaseLocal=true
DataAccess opened => isDatabaseLocal: true
Db initialized
DataAcess closed
Creating BLFacadeImplementation instance with DataAccess parameter
DataAccess opened => isDatabaseLocal: true
>> DataAccess: getProducts=> from= a
DataAcess closed
_____
FROM LAST TO FIRST
8;sukaldeko mahaia;45.0
7;polar kilor erlojua;30.0
6;orbea mendiko bizikleta;225.0
4;imac 27;200.0
3;samsung 42" telebista;175.0
2;salomon mendiko botak;20.0
1;futbol baloia;10.0

_____
FROM FIRST TO LAST
1;futbol baloia;10.0
2;salomon mendiko botak;20.0
3;samsung 42" telebista;175.0
4;imac 27;200.0
6;orbea mendiko bizikleta;225.0
7;polar kilor erlojua;30.0
8;sukaldeko mahaia;45.0
```



El programa busca las ventas que contienen "a" y las muestra primero de la última a la primera (goLast + hasPrevious/previous) y después de la primera a la última (goFirst + hasNext/next).

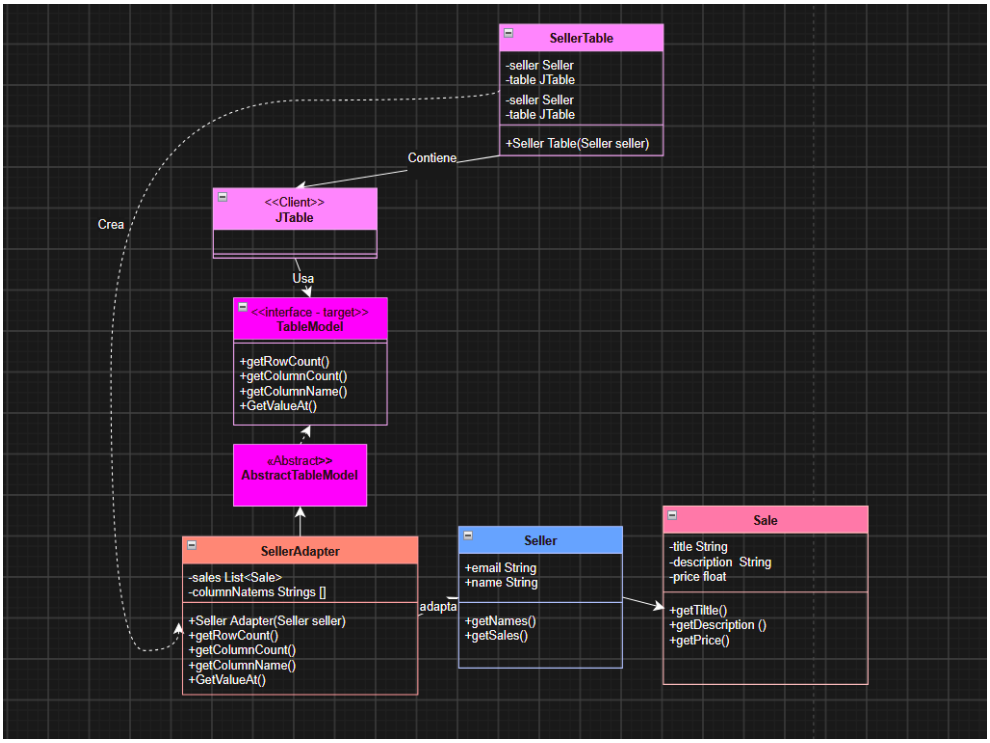
Patrón Adapter

UML y finalidad de cada clase

Se quiere adaptar la aplicación para que muestre todas las ofertas de un vendedor en una JTable. Seller permite acceder a sus ofertas por medio de getSales(), pero JTable solo sabe trabajar con objetos del tipo TableModel. Para resolver esta incompatibilidad de interfaces, se crea la clase SellerAdapter, que hereda AbstractTableModel y permite traducir las peticiones de la tabla, a llamadas sobre el Seller.

Papel de cada clase:

- **TableModel- AbstractTableModel (Target):** una interfaz que necesita la JTable. Tiene los métodos para manejar la tabla: getRowCount, getColumnCount, getColumnName y getValueAt. Usando AbstractTableModel, se puede implementar parcialmente.
- **SellerAdapter (Adapter):** hereda AbstractTableModel y guarda la lista de las ofertas del Seller. Permite devolver una fila para cada Sale organizada en 3 columnas (Title, description y price). Usando getValueAt se obtiene el valor para cada celda llamando a getTitle, getDescription o getPrice.
- **Seller (Adaptee)**: clase del dominio que contiene las ofertas del vendedor, incompatible con JTable
- **JTable (Client):** componente de Swing que permite mostrar la tabla y solicita los datos por medio de la interfaz TableModel.
- **SellerTable:** una ventana Jframe que recibe a un Seller, para envolverlo en un SellerAdapter y así pasarselo a la Jtable.



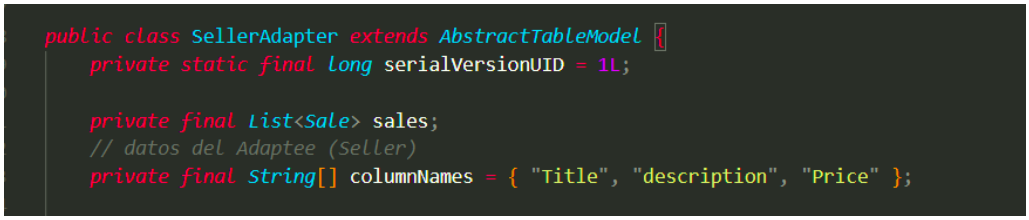
Código antes

Antes no existía forma de mostrar las ofertas de un Seller en una JTable; las tres clases son nuevas.

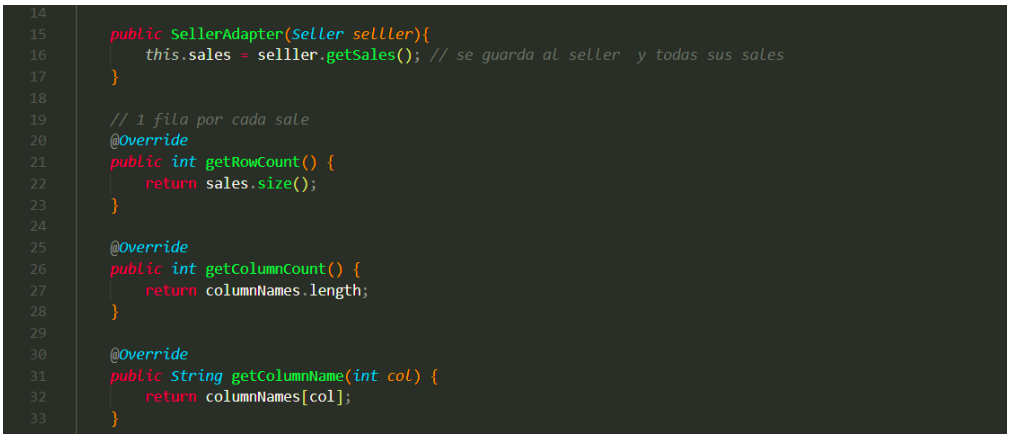
Código después



Usando el patrón de adapter, se crea una clase que funciona como intermediaría para implementar la interfaz que JTable espera. Su trabajo es traducir las peticiones que se hace sobre Seller.



SellerAdapter hereda AbstractTableModel, ya que el adaptador es un TableModel y se le puede pasar una JTable directamente.



Se recibe al seller en el constructor y se guarda una referencia a la lista con todas sus ventas. GetRowCount( ) devolverá el sales.size(), para que cada venta corresponda a una fila en la

tabla.GetColumnCount() y getColumnName(col) define las 3 columnas y su titulo, a partir del array ColumnName.

```
34
35      @Override
36      public Object getValueAt(int row, int col) {
37          Sale s = sales.get(row);
38          switch (col) {
39              case 0:
40                  return s.getTitle();
41              case 1:
42                  return s.getDescription();
43              case 2:
44                  return s.getPrice();
45              default:
46                  return null;
47          }
48      }
49  }
```

GetValueAt(row, col) obtiene la venta de la fila row y según la columna regresará el título, la descripción o el precio. Usando los getTitle(), getDescription(), getPrice() ya definidos en Sale.

Ejecución

```java
package gui;

import businesslogic.BLFacade;
import businesslogic.BLFactory;
import domain.Seller;

public class AdapterMain {
    Run | Debug
    public static void main(String[] args) {
        boolean isLocal = true;
        BLFacade blFacade = new BLFactory().getBusinessLogicFactory(isLoc
        Seller s = blFacade.isLogged(log: "Aitor Fernandez", pass: "aurrer
        SellerTable st = new SellerTable(s); // el enunciado pone "Saller
        st.setVisible(b: true);
    }
}

```

BlFacade.isLogged (log, pass) permite obtener el Seller. Una vez se tiene al seller, se llama a new SellerTable(s) y setVisible(true) muestra la nueva ventana con la tabla rellena con las ofertas pertenecientes al Seller.

